

5 Background services

Android conceptualised the Service class to create application components specifically to handle operations and functionality that should run invisibly, without a user interface. It accords Services a higher priority than inactive Activities, so they're less likely to be killed when the system requires resources. In fact, should the run time prematurely terminates a Service that's been started, it can be configured to restart as soon as sufficient resources become available. In extreme cases, the termination of a Service—such as an interruption in music playback—will noticeably affect the user experience, and in these cases a Service's priority can be raised to the equivalent of a foreground activity.

By using Service, you ensure that your applications continue to run and respond to events, even when they're not in active use. They run without a dedicated GUI, but, like Activities and Broadcast Receivers, they still execute in the main Thread of the application's process. To help keep your applications responsive, you'll learn to move time-consuming processes (like network lookups) into background threads using the Thread and AsyncTask classes.

5.1 Services

Unlike Activities, which present a rich graphical interface to users, the Service class runs in the background—updating your Content Providers, firing Intents, and triggering Notifications. They're the perfect means of performing ongoing or regular processing and of handling events even when your application's Activities are invisible or inactive, or have been closed.

Services are started, stopped, and controlled by other application components, including other Services, Activities, and Broadcast Receivers. If your application performs actions that don't depend directly on user input, Services may be the answer.

Since started Services always have higher priority than inactive or invisible Activities, the only reason Android will stop a Service prematurely is to provide additional resources for a foreground component (usually an Activity). When that happens, your Service will automatically restart when resources become available.

If your Service is interacting directly with the user (for example, by playing music) it may be necessary to increase its priority to that of a foreground Activity. This will ensure that your Service isn't terminated